

CO4-AT2 INDUSTRY PROBLEM

FURTHOSE SAMREEN B

192511164

Q10. Job Sequencing – Profit Maximization

A company must schedule jobs with deadlines and associated profits such that maximum profit is earned

within given time constraints. Analyze how the Job Sequencing with Deadlines problem can be solved using a

Greedy approach. Design a solution by explaining the selection and scheduling strategy. Justify why the

greedy choice leads to optimal profit. Suggest appropriate tools and data structures for implementation.

Analyze time complexity and interpret results in terms of business profit optimization.

ANSWER;

Introduction

Job Sequencing with Deadlines is a well-known **Greedy Algorithm** problem. A company has a number of jobs, where each job has a deadline and a profit. Every job requires one unit of time to complete. The objective is to select and schedule jobs so that they are completed before their deadlines and the **total profit is maximized**.

This problem is useful in business applications where there are limited time slots and many tasks competing for execution.

1. Greedy Approach

The problem is solved using a **Greedy Strategy**.

The basic idea is to give priority to jobs that provide **higher profit**.

The steps are:

1. Sort all jobs in **decreasing order of profit**.
2. Start with the job having the highest profit.
3. Check its deadline.
4. Place the job in the **latest available time slot before or on its deadline**.
5. If no suitable slot is available, skip the job.
6. Repeat this process for all jobs.
7. Add the profits of all scheduled jobs to obtain the maximum profit.

The important greedy rule is:

Select the highest-profit job first and schedule it as late as possible before its deadline.

2. Example

Consider the following jobs:

Job Deadline Profit

J1 2 100

J2 1 50

J3 2 80

Job Deadline Profit

J4 1 40

J5 3 60

First, sort the jobs according to decreasing profit:

Job Deadline Profit

J1 2 100

J3 2 80

J5 3 60

J2 1 50

J4 1 40

The maximum deadline is **3**, so there are three available time slots.

Slot 1 Slot 2 Slot 3

Scheduling J1

J1 has deadline 2 and profit 100.

The latest available slot is Slot 2.

Slot 1 Slot 2 Slot 3

J1

Profit = 100

Scheduling J3

J3 has deadline 2 and profit 80.

Slot 2 is occupied, so we use Slot 1.

Slot 1 Slot 2 Slot 3

J3 J1

Profit = 80

Scheduling J5

J5 has deadline 3 and profit 60.

Slot 3 is available, so J5 is placed in Slot 3.

Slot 1 Slot 2 Slot 3

J3 J1 J5

Profit = 60

Scheduling J2

J2 has deadline 1. Slot 1 is already occupied by J3.

Therefore, J2 is rejected.

Scheduling J4

J4 also has deadline 1. Slot 1 is occupied.

Therefore, J4 is rejected.

3. Final Result

The final schedule is:

Time Slot Job Profit

1 J3 80

2 J1 100

3 J5 60

Therefore,

Maximum Profit = 80 + 100 + 60

Maximum Profit = 240

4. Why Greedy Choice Gives Optimal Profit

The greedy algorithm always considers the job with the **highest profit first**. This is useful because selecting a high-profit job gives a greater contribution to the total profit.

However, the job is not simply placed in the first available slot. It is placed in the **latest available slot before its deadline**.

For example, if a job has a deadline of 3, the algorithm checks:

Slot 3 → Slot 2 → Slot 1

This leaves earlier slots available for jobs with smaller deadlines.

Thus, the greedy method combines two important decisions:

- **Highest profit first**
- **Latest possible slot**

This helps achieve the maximum possible profit while satisfying deadlines.

5. Data Structures and Tools

The following data structures can be used:

Structure

A structure can store the details of each job:

```
struct Job {  
    char id;  
    int deadline;  
    int profit;
```

```
};
```

It stores:

- Job ID
- Deadline
- Profit

Array

An array can store all the jobs and another array can represent the available time slots.

Sorting

Jobs are sorted according to their profit in **descending order**.

Programming Languages

The algorithm can be implemented using:

- C
- C++
- Java
- Python

For a simple implementation, **C using structures, arrays, and sorting** is suitable.

6. Time Complexity

Let the number of jobs be **n**.

Sorting

Sorting the jobs based on profit takes:

$O(n \log n)$

Scheduling

In the basic approach, each job may search through several available slots.

Therefore, scheduling takes:

$O(n^2)$

Overall Complexity

The overall time complexity is:

$O(n^2)$

The space required for storing jobs and time slots is:

$O(n)$

For large systems, a **Disjoint Set Union (DSU)** can be used to find available slots more efficiently.

7. Business Profit Optimization

Job sequencing can be applied in many real-world business situations.

For example:

- Manufacturing orders
- Customer service requests
- Software development tasks
- Delivery scheduling
- Advertising campaigns
- Machine scheduling
- Project management

A company may have many jobs but only a limited number of working hours. Completing every job may not be possible. Therefore, the company should select the jobs that provide the highest profit while meeting their deadlines.

Using the Greedy Algorithm helps the company:

1. **Increase total revenue**
 2. **Prioritize high-value jobs**
 3. **Utilize available time efficiently**
 4. **Reduce unnecessary delays**
 5. **Meet important deadlines**
 6. **Improve overall resource utilization**
-

8. Advantages

- Simple and easy to understand.
 - Easy to implement using arrays and structures.
 - Provides high profit by prioritizing valuable jobs.
 - Efficient for basic job scheduling problems.
 - Uses limited time slots effectively.
 - Useful in practical business scheduling applications.
-

9. Limitations

The standard Job Sequencing problem assumes that every job requires exactly **one unit of time**. It also assumes that jobs are independent and that a job earns its profit only when completed before its deadline.

If jobs have different execution times, dependencies, or multiple resources, more advanced scheduling techniques may be required.

10. Conclusion

Job Sequencing with Deadlines is an important application of the **Greedy Algorithm**. The jobs are first arranged according to decreasing profit. Each job is then placed in the latest available slot before its deadline.

For the given example, the selected jobs are:

J3 → J1 → J5

and the total profit is:

$$80 + 100 + 60 = 240$$

Therefore, the **maximum profit is 240**.

The Greedy approach is useful because it gives priority to high-profit jobs while preserving earlier time slots for jobs with smaller deadlines. Hence, it provides an effective method for **maximizing business profit under limited time constraints**.